2025-2026 Fall Semester

COMP413

Internet of Things

Final Project Report

Instructor

Dr. Abdulkadir KÖSE

Subject

Illegal Parking Detection

Submission Date

08.01.2026

Submitted by

Group 2

Syed Muhammad Mojiz Ali ZAIDI - 2311051806

Selin ERYAŞAR - 2111051051

Mohamed MOSTAFA - 2211011095

Güleser KABA-110510234

ILLEGAL PARKING DETECTION SYSTEM USING IOT AND TINYML

Abstract

The critical issue confronting cities and leading to traffic congestion, safety risks, and ineffective utilization of public infrastructure facilities is illegal parking. This article introduces an IoT-based system of illegal parking detection using edge computing and TinyML in real-time vehicle presence detection. Every illegal parking zone will be allocated a complete autonomous ESP32 IoT module with a camera and a distance sensor. A machine learning model detects the presence of any vehicle in real time and communicates with the server via an MQTT-based communication mechanism. In short, the system is amenable to expansion and, therefore, attains low latency of illegal parking detection under a smart city environment.

1. INTRODUCTION

The rising number of people in urban areas and vehicles has exacerbated parking problems in current city infrastructure. Illicit parking practices contribute to route disruption of vehicles for emergency routes and pedestrian paths. Accordingly, this phenomenon inhibits traffic movement and causes inefficiency in the performance of urban transport infrastructure. It was shown in reference [1] that a lack of efficient parking practices results in deteriorated travel time utilization and infrastructure use in urban towns.

Conventional parking control solutions mainly use manual observation or camera observation in a fixed mode. In addition, such conventional solutions mainly involve intensive human effort and high resource costs in implementation, and also face difficulties in implementation on a distributed basis in a large city environment itself. Recently emerging works clearly specify and emphasize the fact that solutions related to fixed observation solutions in a centralized manner can mainly introduce intense latency and high usage of the network, and also lack adaptability related to dynamic environments in a city [2].

IoT-based smart parking systems incorporate embedded sensors and microcontrollers integrated with wireless communication protocols for monitoring parking spaces in real time. Presently, vehicle detection systems have been implemented by incorporating ultrasonic as well as proximity sensors together. Light-weight communication protocols such as MQTT provide efficient data transmission to a central server [3].

The most recent research development in parking surveillance is integrating computer vision and embedded AI as a way of improving detection accuracy. Edge-based machine learning provides vehicle classification on resource-constrained devices directly, thereby reducing constant video streaming and cloud-side processing [4]. Among these, TinyML frameworks run neural network inference on microcontrollers by providing low latency, lower bandwidth usage, and better system scalability [5].

To address the mentioned issues and based on the developments that trigger the concept of this research, this study proposes an IoT-based illegal parking detection system incorporating ESP32-based edge nodes, the use of sensors for parking detection, vehicle verification using TinyML machine learning, and real-time interactions between the edge devices and the centralized server. The proposed system can efficiently make decisions through edge intelligent computation and send particular events to its server.

The project aims to ensure the achievement of **SDG Goal 11: Sustainable Cities and Communities** through its efficient solutions in urban mobility and safety related to intelligent illegal parking detectors. The proposed project will remove city traffic congestions and ensure that pedestrian and emergency lanes are not blocked in urban areas.

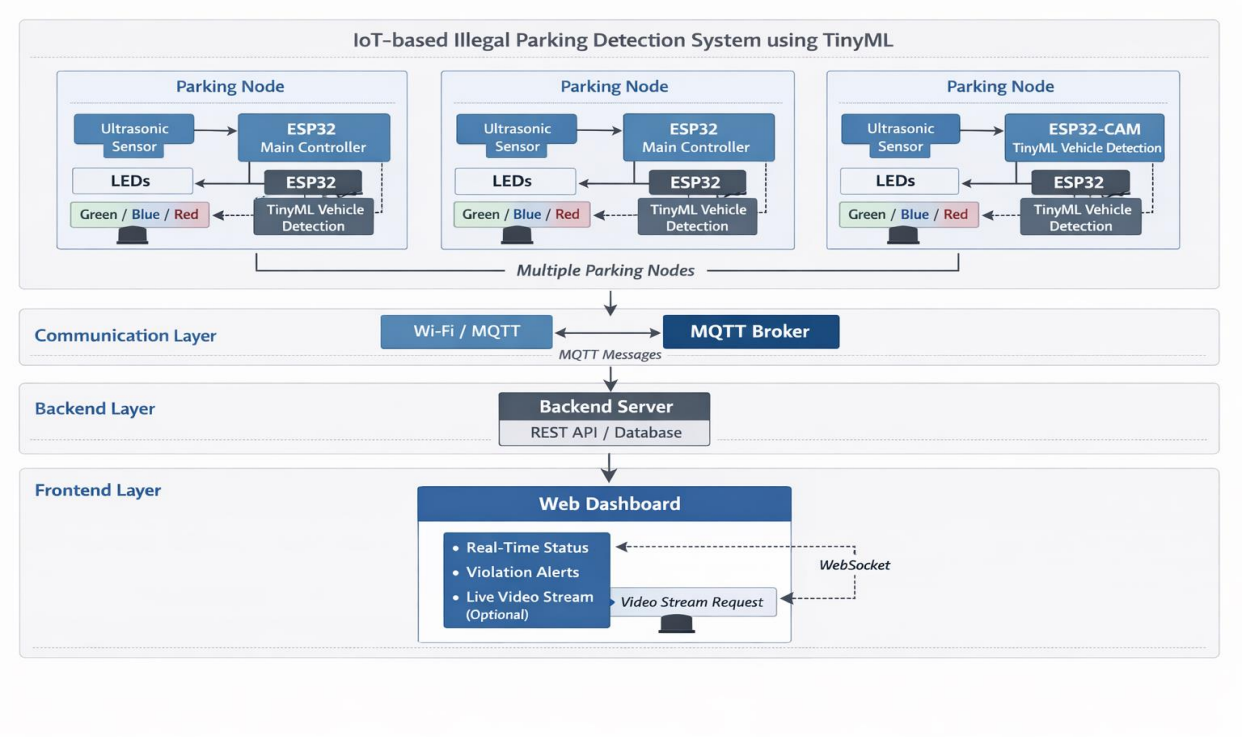


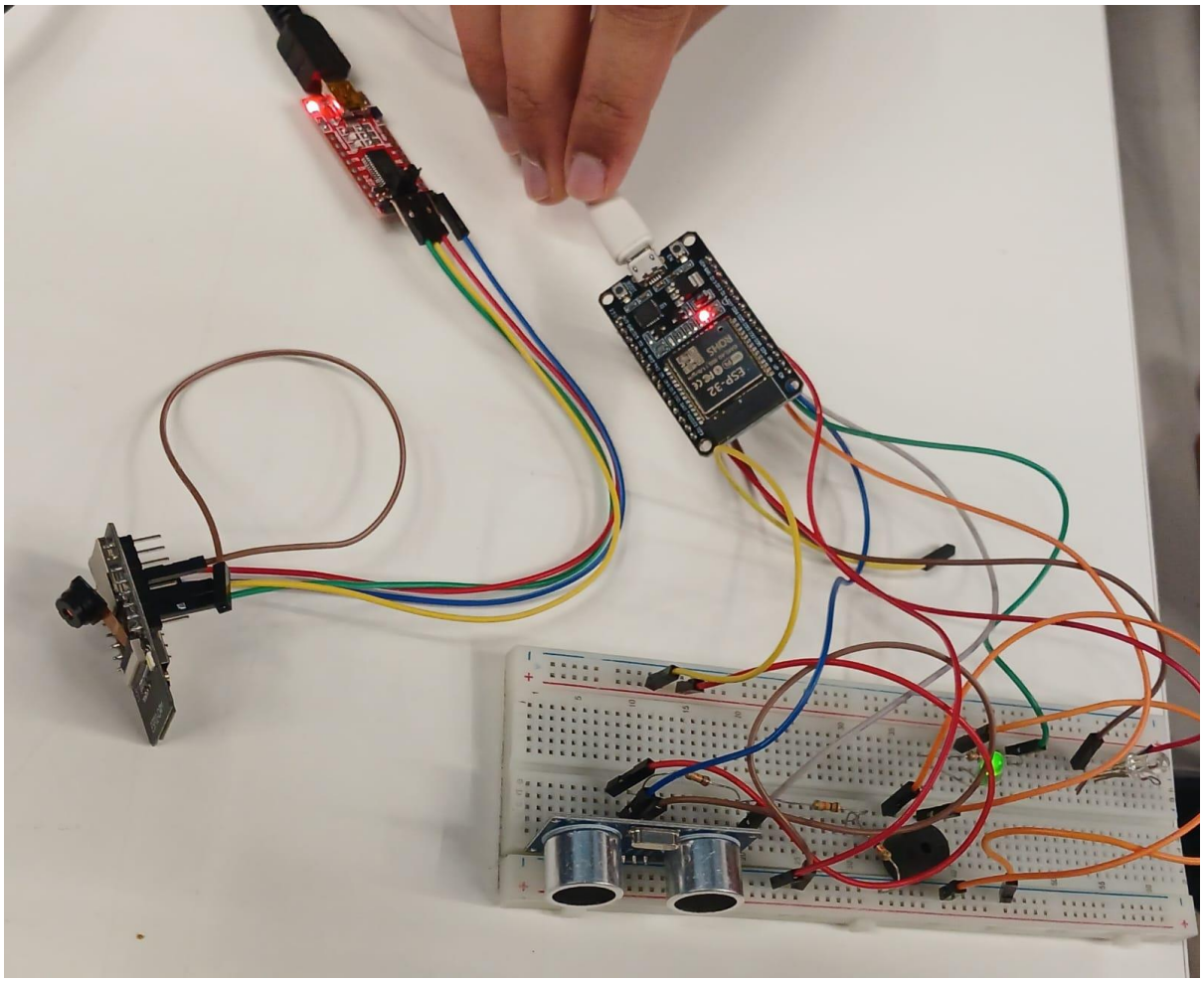
Fig. 1. Overall architecture of the IoT-based illegal parking detection system.

2. SYSTEM MODEL

2.1. Hardware Design

Every illegitimate parking spot is also watched over by a self-contained unit powered by an ESP32 microcontroller, designed for use as a smart edge node. The ESP32 microcontroller was chosen for its low cost, the presence of Wi-Fi, enough processing power, as well as its good compatibility with communication standards in IoT applications, TinyML, among others, for distributed sensing application in smart cities.

The hardware design has a modular edge computing architecture with a concentration of sensing, decision making, artificial intelligence inference, and actuation in a single location near the monitoring site.



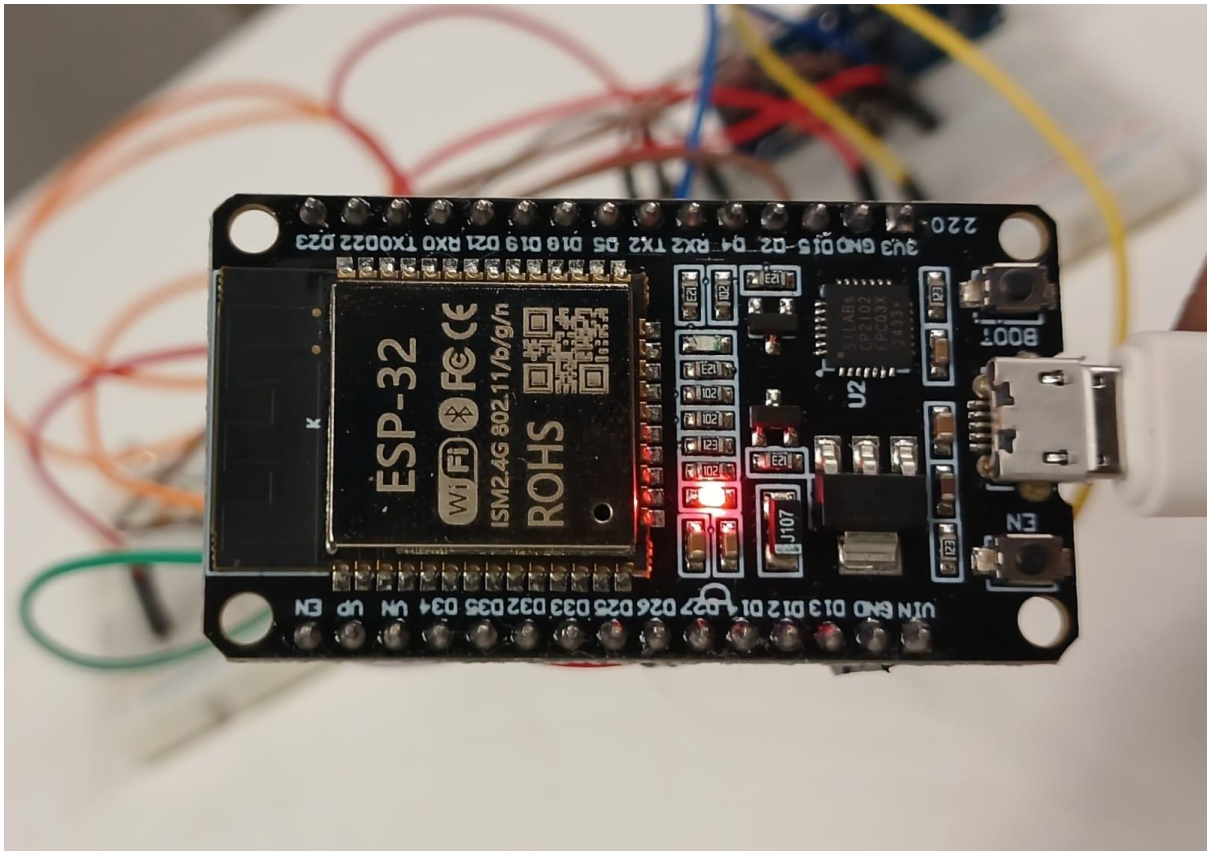
The hardware setup consists of the following main components:

ESP32 Main Module

The ESP32 acts as the central controller of the system. It is responsible for:

- Continuous acquisition of sensor data
- Executing the system decision logic
- Managing Wi-Fi and MQTT communication
- Sending commands to actuators (LEDs and buzzer)
- Exchanging verification data with the ESP32-CAM

Its dual-core processor allows reliable handling of both sensor processing and network communication.

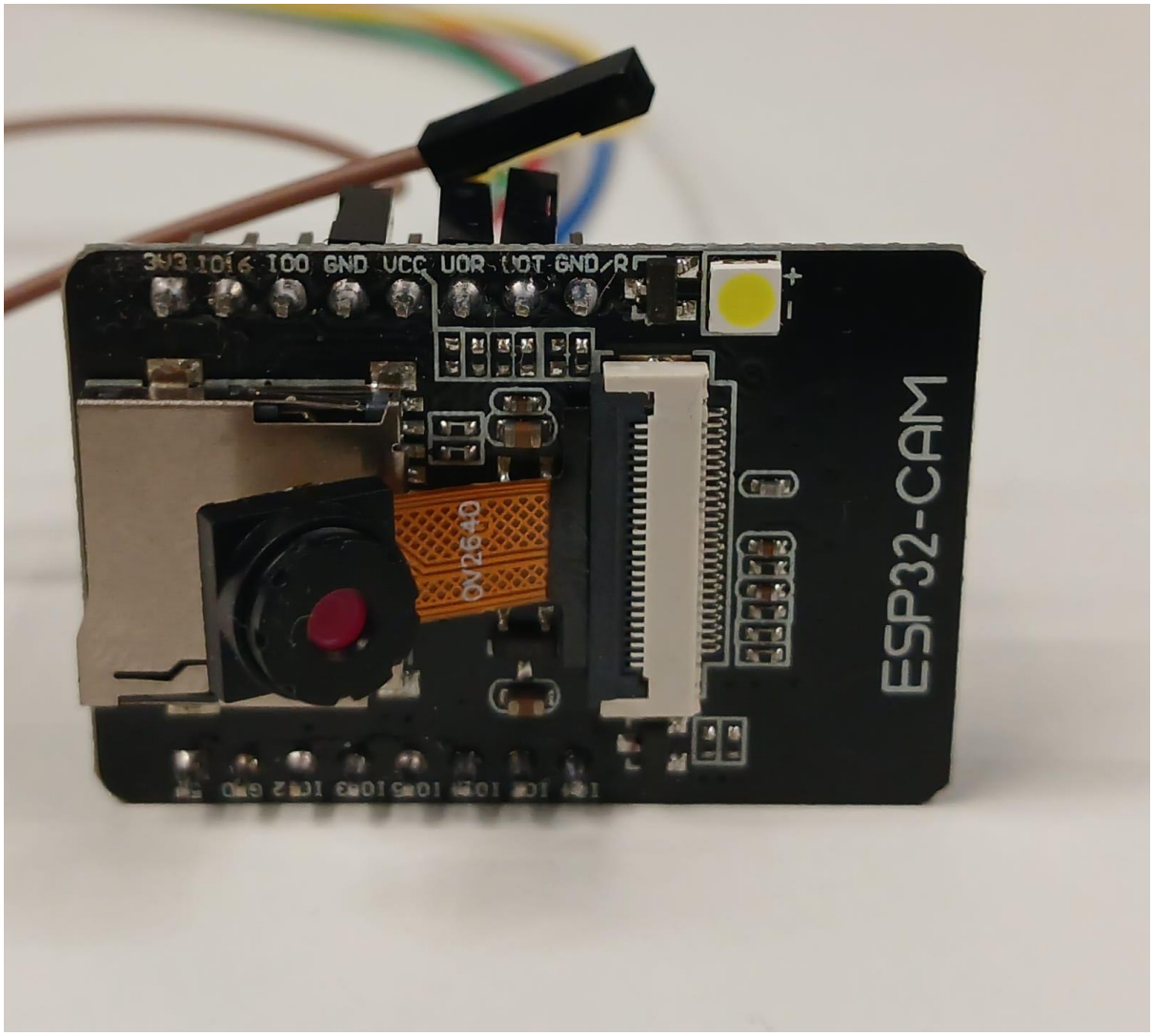


Distance / Thermodynamic Sensor

However, the sensor distance constantly monitors the proximity of objects within closed parking dimension. Exceeding a threshold in the measured value makes obstructions possible. This is the first-level physical detection that allows the system to maintain energy efficiency by switching on advanced processes only where necessary.

ESP32-CAM Module

The ESP32-CAM module is responsible for capturing image frames of the observed area and running an inference model using TinyML. The inference model is used to decide whether the classified object is that of a vehicle. The ESP32-CAM module sends inference results rather than transmitting unnecessary data in the form of videos since classification has been done locally. The ESP32-CAM sensor will only stream the inference results and videos in case of an observed violation.



LED Indicators

Visual indicators are used to represent system states:

- Green LED – Idle / No vehicle
- Blue LED – Object detected / Verification in progress
- Red LED – Violation confirmed

This allows for rapid recognition of status on site and facilitates debugging and maintenance.

Buzzer

The buzzer gives a sound signal when a violation has been detected. It is automatically triggered by the ESP32 and can be turned off remotely using commands on the dashboard via MQTT.

Operational Flow at Hardware Level

The distance sensor keeps on calculating the range of the monitored area. As soon as any object enters within the range, the ESP32 triggers the ESP32-CAM module to record images. As soon as the model recognizes the vehicles, if it has entered within the set time range, then it marks it as an illegal parking

case. Even then, the red LED starts lighting up, as well as the buzzer starts sounding, with the violation message sent to the server.

Each hardware component is powered and managed by the ESP32, making it a small, low power, self-sustained edge device that can run in real-time continuously.

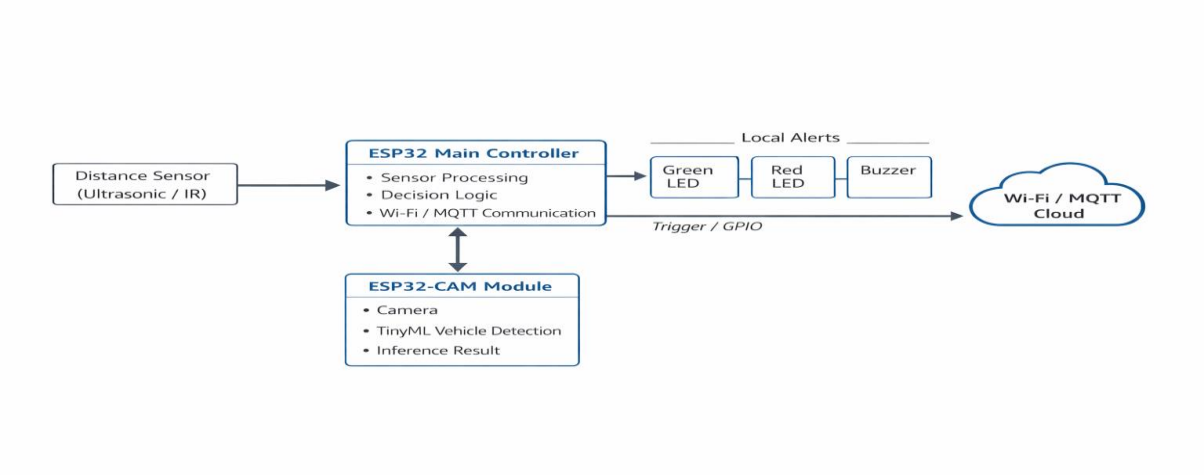


Fig. 2. Overall hardware design.

2.1.1. Hardware Control Source Code

The following code fragment demonstrates the core hardware functionality implemented on the ESP32. It handles sensor acquisition, actuator control, and serial communication with the ESP32-CAM. The program continuously monitors the distance sensor, controls LED indicators, and activates the buzzer when a potential violation condition is detected.

```
#include <WiFi.h>
#define TRIG_PIN 32
#define ECHO_PIN 33
#define RED_LED 14
#define GREEN_LED 12
#define BUZZER 25

long duration;
float distance;
```

```
void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);
}

float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH);
  return duration * 0.034 / 2;
}

void loop() {
  distance = readDistance();
  Serial.print("Distance: ");
  Serial.println(distance);

  if (distance < 50) {  // Object detected
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);

    // Signal ESP32-CAM for image capture / ML inference
```

```
    Serial.println("OBJECT_DETECTED");

  } else {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
  }

  delay(500);
}
```

Code Functionality Explanation

- The ultrasonic sensor continuously measures the distance to objects in the monitored area.
- If the measured distance drops below the defined threshold, the ESP32 identifies a possible parking event.
- The system activates the red LED and buzzer and sends a trigger signal to the ESP32-CAM.
- If no object is present, the system remains in idle mode with the green LED active.

2.2. Software Design

The software architectural design used in the illegal parking detection system takes a distributed edge-to-server architectural approach, whereby functions such as critical timing and intelligence are implemented in the ESP32 modules, while coordination and visualization happen in the central servers. This enhances scalability and real-time monitoring for several illegal parking sites.

The following five main layers can be identified in this software system:

1. TinyML inference layer -ESP32 CAM
2. Edge control and decision layer - ESP32 main module
3. Communication Layer -MQTT, HTTP, WebSocket
4. Backend Processing and DB Layer
5. Frontend Visualization and Control Layer

2.2.1. TinyML Vehicle Detection Model

In order to increase the reliability of detection and reduce the issue of false alarms, the TinyML model runs on the ESP32-CAM module. In fact, the TinyML model's main task is to identify vehicles from other irrelevant objects like pedestrians, shadows, or environmental noise. The model recognizes the captured images into two groups: when there is a vehicle and when there is no vehicle.

Each captured frame is processed locally by the TensorFlow Lite Micro inference engine running on the ESP32-CAM. The model predicts a confidence value between 0 and 1. If the confidence value surpasses a threshold value, the object is then deemed to be a vehicle. This AI-enabled validation process ensures that the violation processing stage is triggered by a valid detection.

The benefits of executing the model within the device include:

- Since the inference is carried out locally, there is low latency.
- Less bandwith is utilized, as unprocessed photos are no longer being streamed
- Privacy enhancement due to personal information being held on the edge
- Improved system stability and robustness because the scanning process doesn’t require a continuous connection with the server

The ESP32-CAM transmits live video only when there is an actual violation and when the dashboard user wants to have visual proof.

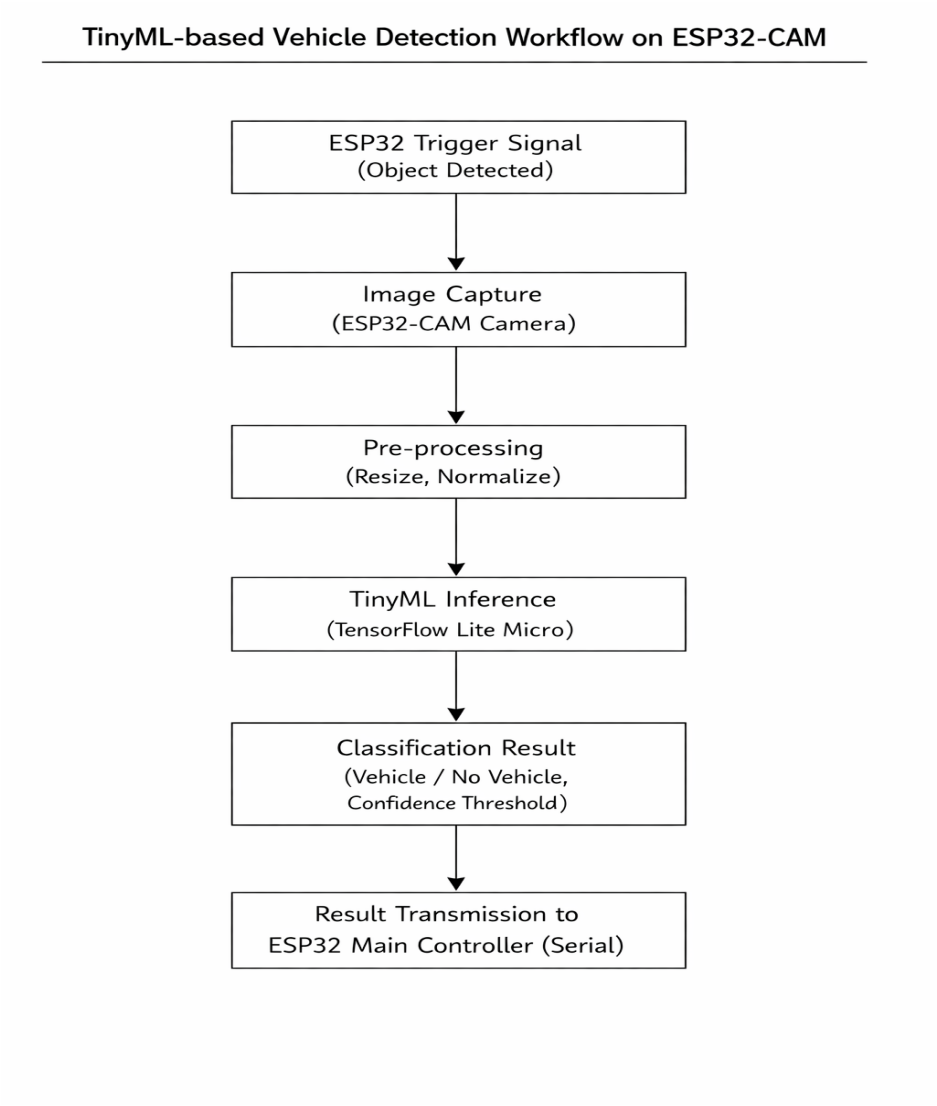


Fig. 3: TinyML-based vehicle detection workflow on ESP32-CAM.

2.2.2. Detection and Decision Logic

The main ESP32 drives a finite-state decision engine that manages sensor information, TinyML feedback, and system responses. This system has four major state transitions, which are idle, object detected, vehicle confirmed, and violation.

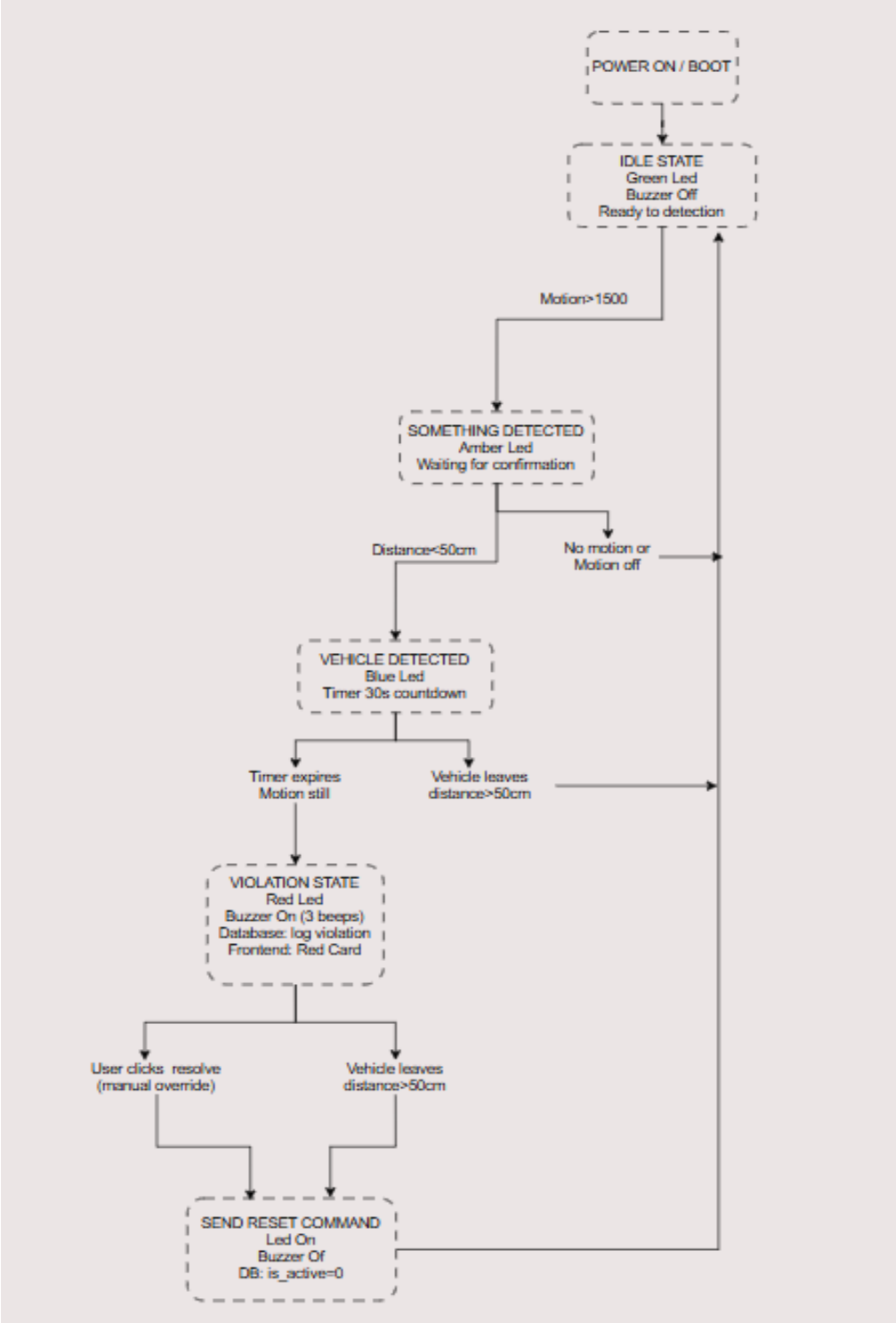
Firstly, it continues to be in the idle state as it continuously monitors the distance and thermodynamic sensor values. However, upon detecting any obstacle within the defined threshold, it switches to an object-detected state and requests ESP32-CAM to do image analysis.

If this condition is met and the TinyML model verifies that it is a vehicle, the system enters vehicle-validated status and starts a timer. The timer prevents a violation when the vehicle halts for a short period of time because, if it stays longer than allowed, it moves to the violation state.

In the violation state:

- The red LED is switched on
- The buzzer alarm is triggered
- A violation message will be sent using MQTT
- Backend will update the dashboard and log the event.

Also, this multi-layer logic ensures the triggering of a breach notification only when both physically detected and AI-verified conditions are met. Moreover, accuracy levels are substantially improved.



2.2.3. Communication Architecture

The system is based on the usage of a hybrid communication architecture, operating over a Local Wi-Fi network, which combines MQTT, HTTP, and WebSocket technologies.

- MQTT is utilized for lightweight and real-time communication between ESP32 nodes, an MQTT agent server, and an ESP32 backend involved in data transferring as follows:
  - Vehicle detection results
  - Violation alerts
  - Node status updates
  - Remote control commands - buzzer silencing

- By using the HTTP REST APIs, the following data exchange is accomplished:

  - Violation reporting
  - Session logging
  - Database updates
  - System configuration operations

- The WebSocket (Socket.IO) connections are used for real-time updates pushed from the backend side towards the dashboard user interface.

This multi-protocol approach presents the possibility of achieving high responsiveness to real-time monitoring and ensuring reliable management of data during long-term system functioning.

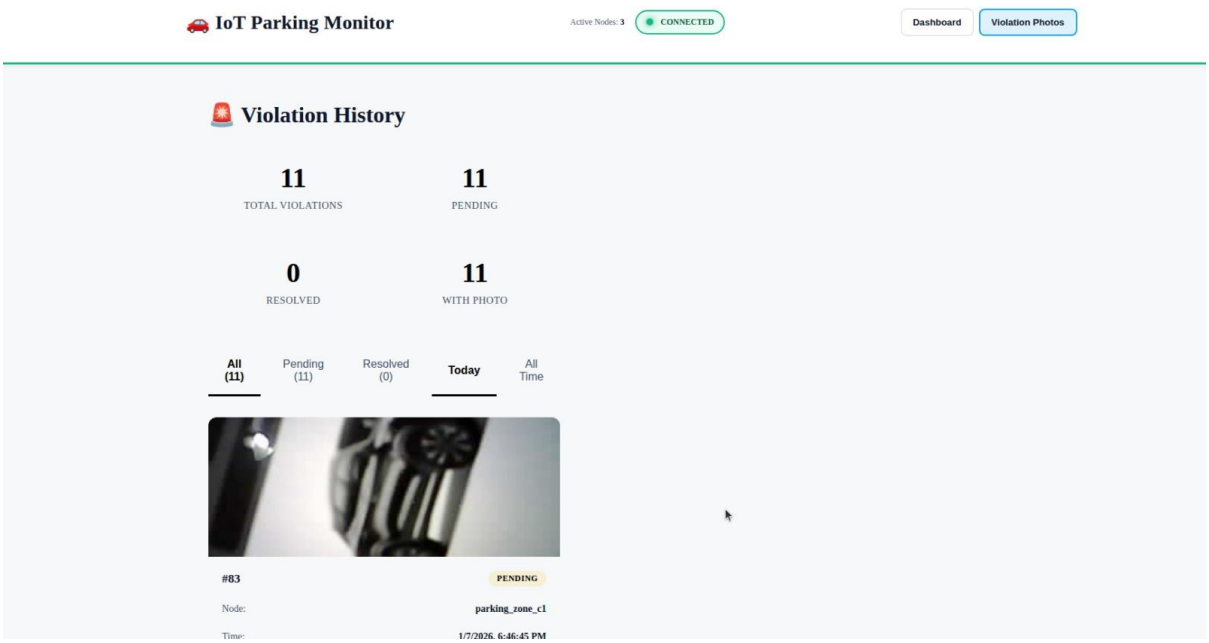
2.2.4. Dashboard Application

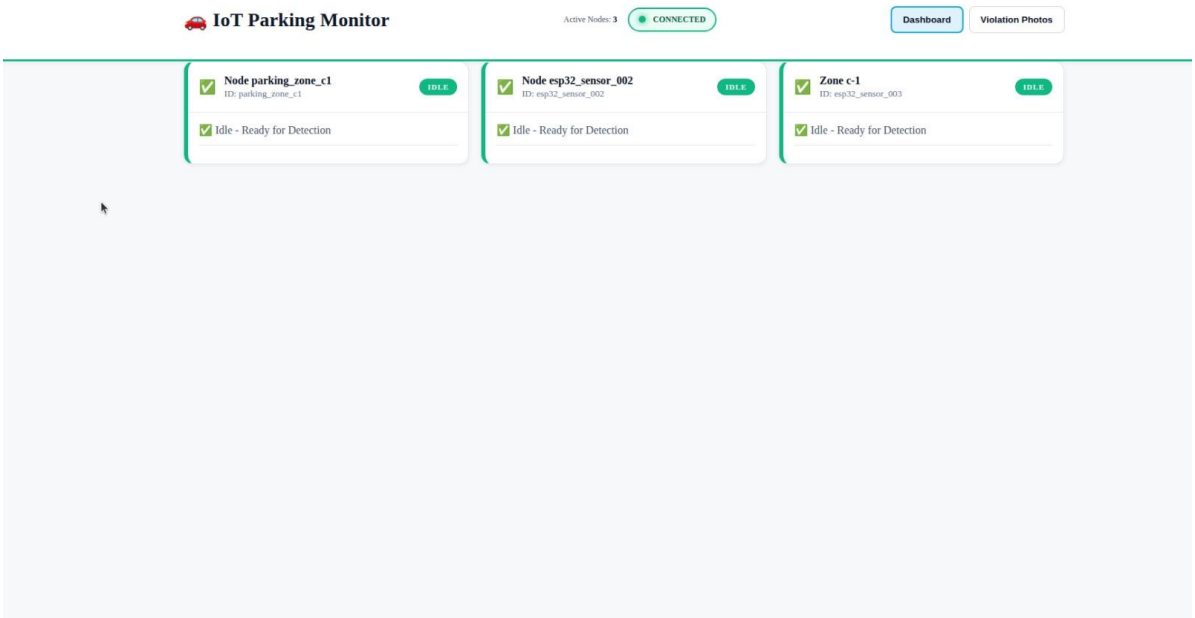
A web-operated dashboard was also developed to create an overall operation interface. The dashboard was designed to display all the deployed modules as a series of cards, with each card representing an illegal parking spot, and these cards change color dynamically to indicate the statuses.

When a violation occurs:

- The corresponding module card turns **red**
- A **“View Video”** button becomes available
- The event is logged and timestamped
- The live camera feed can be accessed for verification

The live system updates are received via WebSocket communication to the dashboard. Besides visualization, the dashboard offers remote controlling functionalities to the operators, which silence buzzers, acknowledge violations, and monitor historical activity.





🚨 Violation History

1	1
TOTAL VIOLATIONS	PENDING
0	**1**
RESOLVED	WITH PHOTO

All (1) | Pending (1) | Resolved (0) | **Today** | All Time

Violation evidence

#73 **PENDING**

Node:	**parking_zone_c1**
Time:	**1/7/2026, 5:59:34 PM**
Type:	**parking_violation**
Photo Size:	**0.1 KB**

2.2.5. Integration of Edge Intelligence

The integration of TinyML assists in the transformation of the entire system from the conventional solution with sensors to an intelligent system of edge computing. Instead of transmitting raw video data through the channel, only the inference data or the status/violation events are transmitted.

This design provides several benefits:

- Reduced network congestion
- Faster response time
- Lower server workload
- Improved scalability

Since each node ESP32 operates in an autonomous manner, it is easy to expand it to handle dozens of illegal parking areas. In future implementation, it is easy to introduce a gateway module that aggregates the nodes, forming city-scale smart enforcement infrastructure.

3. RESULTS AND DISCUSSION

Several trials were conducted on campus to evaluate the performance characteristics, reliability, and real-time functionality of the illegal parking system.

The range sensors and thermodynamic sensors were able to detect the presence of objects in the prohibited area and trigger the first stage of the detection pipeline. The sensors were able to deliver real-time readings, and this enabled the rapid detection of potential parking events. However, the sensors faced the challenge of possible false alarms from pedestrians and noise. This problem was solved using the TinyML solution for vehicle verification.

The deployed TinyML model on ESP32-CAM worked reliably for vehicle classification. With on-device inference, the system correctly identified vehicle objects from non-vehicle objects, hence reducing false alarms by a substantial margin.

Only the detections confirmed by the model could proceed to the violation workflow. This validation stage significantly enhanced robustness and avoided unnecessary alarms in the system.

The finite state decision logic was running on the primary ESP32 chip and functioned well. Passing and exiting transitions to/from the idle, object detected, vehicle validated, and violation state were reliable and reproducible. Neither temporary halts nor sustained presence sparked a violation state with the Timing Function, although both conditions produced a confirmed violation state as expected. When a violation was detected, the red alarm LED and buzzer sounded properly to alert the user.

Communication performance was evaluated using the MQTT protocol running on top of the local Wi-Fi. The ESP32 nodes illustrated the delivery of detection data, system status, and violation events

from the nodes to the MQTT broker. The backend system received the notifications with minimal delay and propagated them further to the dashboard. The test cases concerning the update of the dashboard starting a few seconds after each confirmed violation showed responsiveness. Remote control commands, including the silencing of the buzzers too, proved the reliability of the communication from the dashboard to the ESP32 modules.

The backend and database interface recorded all system events, such as sensor events, vehicle confirmations, violation dates, and resolution responses, which is useful for traceability purposes, historical records included. Data loss was not noted during the series of test cycles, which is a clear indication that the interaction between the messaging service and the database service is well integrated.

The dashboard app was able to show the actual status of all modules deployed in real time. Each parking spot was represented as a different card, and any change in status was readily observable from the change in colors. On observation of any infringement, the respective module card turned red, and an option for “View Video” appeared. The video streaming feature allowed operators to monitor infringement in videos, thereby enhancing the usability of this system.

It indicated overall stability with respect to operation, low latency in the response times, and consistency with regards to accuracy in detection. Thus, physical sensing, along with TinyML validation and a decentralized logic for decision, made a remarkably robust approach for the detection of illegal parking. The result validates the effectiveness of the proposed architecture for real-time parking and that it can support more than one parking lot within a smart city scenario

4. FUTURE WORK

Despite its capability to perform effectively and accurately through IoT technology for the detection of illegal parking, there are still areas for improvements that can be adopted for future development.

One of the major extensions of the work is the incorporation of automatic license plate recognition. This is because the system would be able to not only detect vehicles but also recognize their plates. This would help the system to allow the identification of offending vehicles and the subsequent issuance of fines or warning to the offenders.

Another crucial area for future improvements would be to migrate from a local deployment to a cloud-based and multi-location framework. By leveraging cloud platforms for hosting backend services, it would be possible to enable deployment at a city level or even inter-city level.

In addition, one could extend the communication infrastructure by incorporating long-range wireless communication technologies such as LoRaWAN and/or NB-IoT. This would enable deployment within areas that do not have Wi-Fi coverage and would enable large-scale smart city deployment with low power consumption.

Finally, future research could involve the use of data analytics and predictive modeling based on the historical violation data that has been gathered. Machine learning algorithms may be employed to analyze patterns of parking behavior to pinpoint high risk areas of violations to inform urban planning. Such information will revolutionize the system from a response-driven monitoring system to a pro-active traffic control system.

In general, these future improvements would increase the system's intelligence, scalability, and real-world effectiveness, thus making the system a more integrated solution for intelligent urban transport management.

5. SOURCE CODE

Full source code and system documentation can be found at:

https://github.com/zaidi14/comp413-g2-illegal-parking-monitor

REFERENCE LIST

[1] D. Shoup, The High Cost of Free Parking, Chicago, IL, USA: Planners Press, 2017.

[2] A. Al-Turjman and M. Malekloo, “Smart parking in IoT-enabled cities: A survey,” Sustainable Cities and Society, vol. 49, pp. 1–13, 2019, doi: 10.1016/j.scs.2019.101608.

[3] S. Idris, E. Tamil, N. M. Noor, Z. Razak, and K. W. Fong, “Parking guidance system utilizing wireless sensor network and ultrasonic sensor,” Information Technology Journal, vol. 8, no. 2, pp. 138–146, 2009.

[4] M. Ammar, G. Russello, and B. Crispo, “Internet of Things: A survey on the security of IoT frameworks,” Journal of Information Security and Applications, vol. 38, pp. 8–27, 2018.

[5] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, Sebastopol, CA, USA: O’Reilly Media, 2019.